Medical Clinic Appointment Management System

Project Group Members

1. Jana Sharaf Al-shaikh
2. Waad Sami
3. Joori Faisal Al Salmi
4. Reval Hassan
5. Aryam Alsaedi

Distribution of Duties

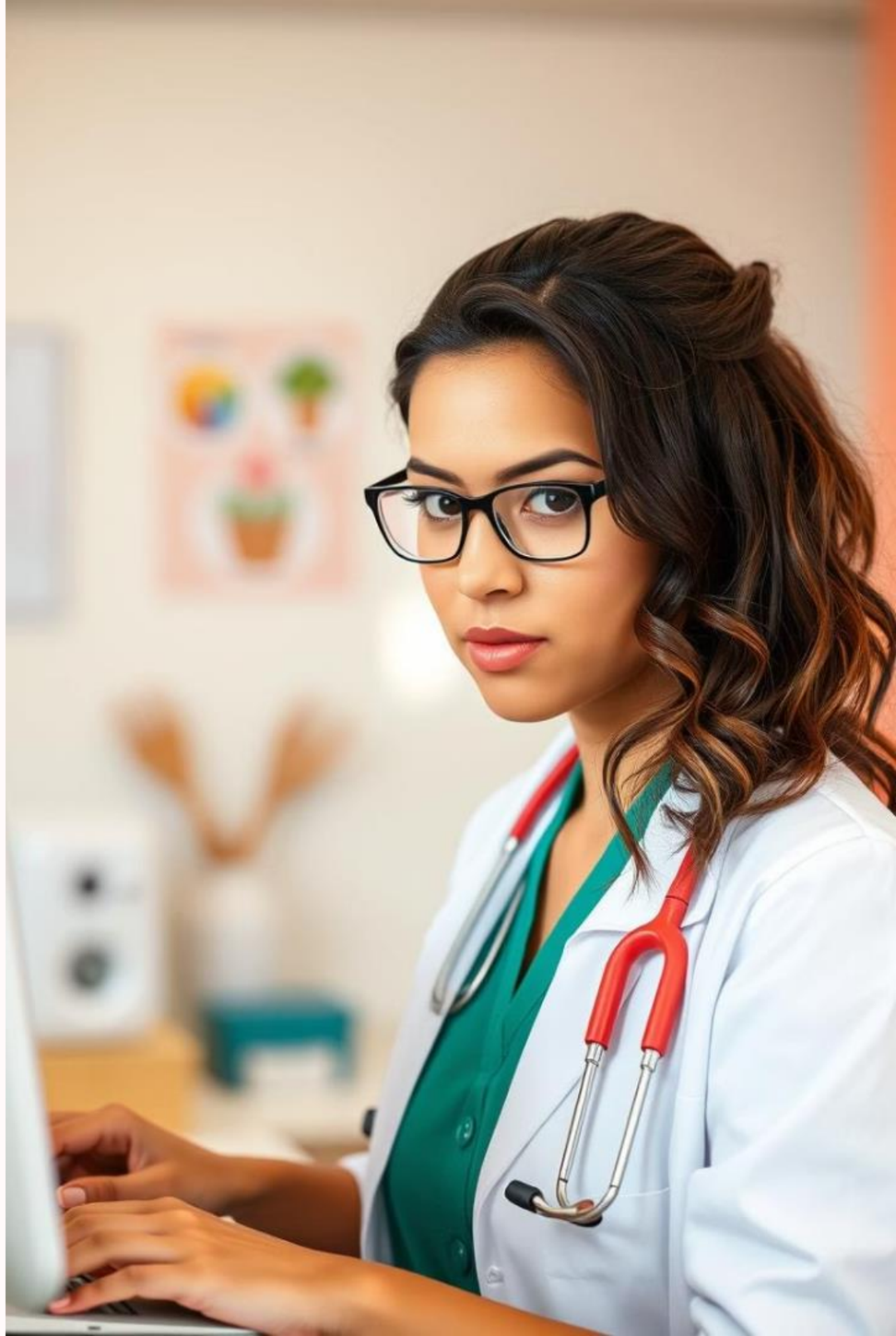
Student	Tasks
Student 1	• **ER Diagram and Schema Design**
Student 2	• **SQL Implementation (Tables & Data)**
Student 3	• **Queries and Stored Procedures**
Student 4	• **Documentation & Final Report**

Introduction

Many clinics currently rely on manual and paper-based systems for managing appointments and patient records. These outdated methods often lead to issues such as overlapping bookings, lost or misplaced records, and significant time wasted on coordination among staff members.

Such inefficiencies not only compromise patient satisfaction but also reduce overall clinic productivity and the ability to provide timely care. Recognizing these challenges, our project is focused on developing an automated appointment scheduling and data management system tailored specifically for medical clinics.

This new solution aims to streamline the entire process, minimize errors, and improve communication between patients and clinic staff. Ultimately, it will enhance operational efficiency, free up valuable staff time, and provide a more reliable and user-friendly experience for both patients and healthcare providers.



DESCRIPTION OF THE PROBLEM:

Manual Scheduling Issues

Leads to errors and frequent double bookings.

Time Consuming

Staff spend excessive time managing appointments.

Tracking Difficulties

Hard to monitor patient history and preferences.

Limited Analytics

Insufficient data for improving operations and wait times.

QUANTIFYING THE PROBLEM: Impact on Clinic Performance

High Costs

Increased administrative overhead due to inefficiencies.

Patient Dissatisfaction

Long waits and scheduling errors cause complaints.

Lost Revenue

Missed appointments reduce earnings significantly.

Staff Strain

Overworked staff morale drops under pressure.

Benefits of Solving this Problem

Fewer Scheduling Errors

Reduce double bookings by significant percentage.

Shorter Wait Times

Decrease wait by optimized appointment flows.

Lower No-Shows

Automated reminders increase attendance rates.

Staff Efficiency

Time saved allows focus on patient care.

Data-Driven Decisions

Improved resource allocation through insights.

OPPORTUNITY

Centralized Management

Unified platform to oversee appointments and patient data.

Automation

Scheduling automation reduces errors and conflicts.

Better Communication

Improves interactions between staff and patients.

Data Insights

Advanced analytics for operational improvements.

FEATURES: How the New System Solves These Problems

Automated Scheduling

Smart booking and reminders.

EHR Integration

Seamless patient record access.

Real-Time Updates

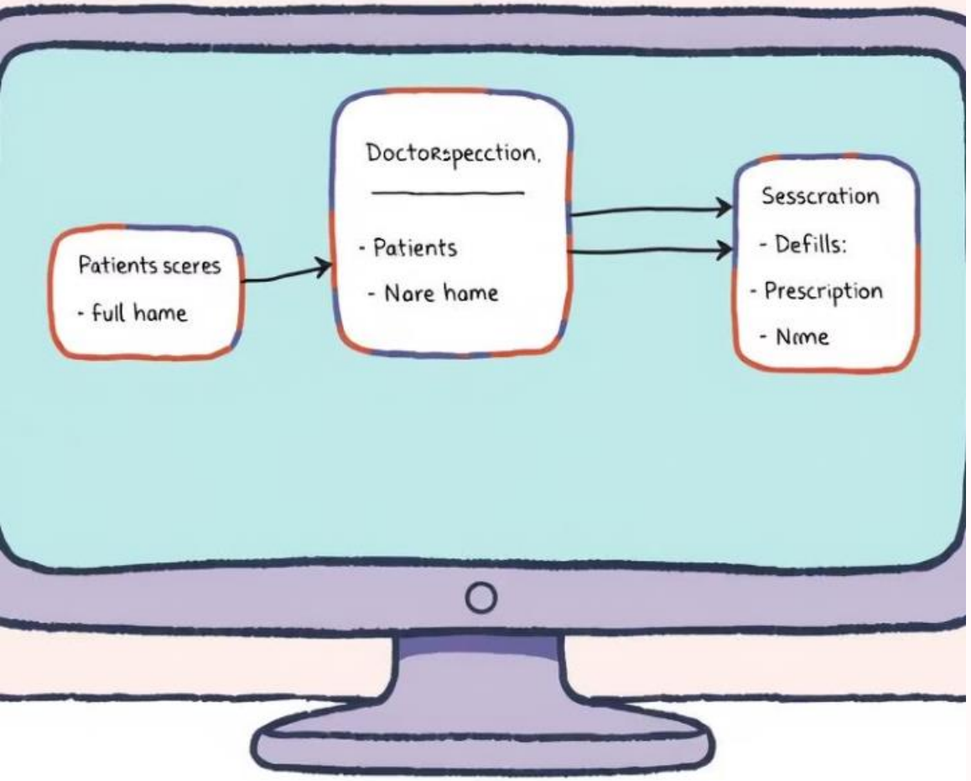
Current availability at a glance.

Customizable Appointments

Flexible types and durations.

Analytics Dashboard

Insightful reports for decisions.



INITIAL LIST OF ENTITIES: Database Design

Patients	Patient ID, Name, Contact, Medical History
Appointments	ID, Date, Time, Patient ID, Doctor ID, Type
Doctors	ID, Name, Specialty, Availability
MedicalRecord	ID, Diagnosis,TreatmentRecordDate
Appointment	ID, Data, Status

Initial Entities (Tables)

1. PatientRepresents individuals receiving medical care.
   - Attributes:
     - PatientID (Primary Key)
     - Name
     - Gender
     - BirthDate
     - ContactNumber
2. DoctorRepresents medical professionals working at the hospital or clinic.
   - Attributes:
     - DoctorID (Primary Key)
     - Name
     - Specialization
     - ContactNumber
     - DepartmentID (Foreign Key → Department)
3. DepartmentRepresents different medical or administrative divisions.

- Attributes:
  - DepartmentID (Primary Key)
  - Name4.

4. MedicalRecordStores the medical history and diagnosis of patients.
   - Attributes:
     - RecordID (Primary Key)
     - PatientID (Foreign Key → Patient)
     - Diagnosis
     - Treatment
     - RecordDate

5. TreatsTracks treatment activities conducted by doctors on patients.

- Attributes:
  - TreatID (Primary Key)
  - DoctorID (Foreign Key → Doctor)PatientID (Foreign Key → Patient)
  - TreatmentDateNotes

6. PrescriptionStores information about prescribed medication.

- Attributes:
  - PrescriptionID (Primary Key)
  - PatientID (Foreign Key → Patient)
  - DoctorID (Foreign Key → Doctor)
  - Medication
  - Dosage
  - PrescriptionDate

7. AppointmentManages patient appointments with doctors.

- Attributes:
  - AppointmentID (Primary Key)
  - PatientID (Foreign Key → Patient)
  - DoctorID (Foreign Key → Doctor)
  - Appointment
  - Date
  - Status

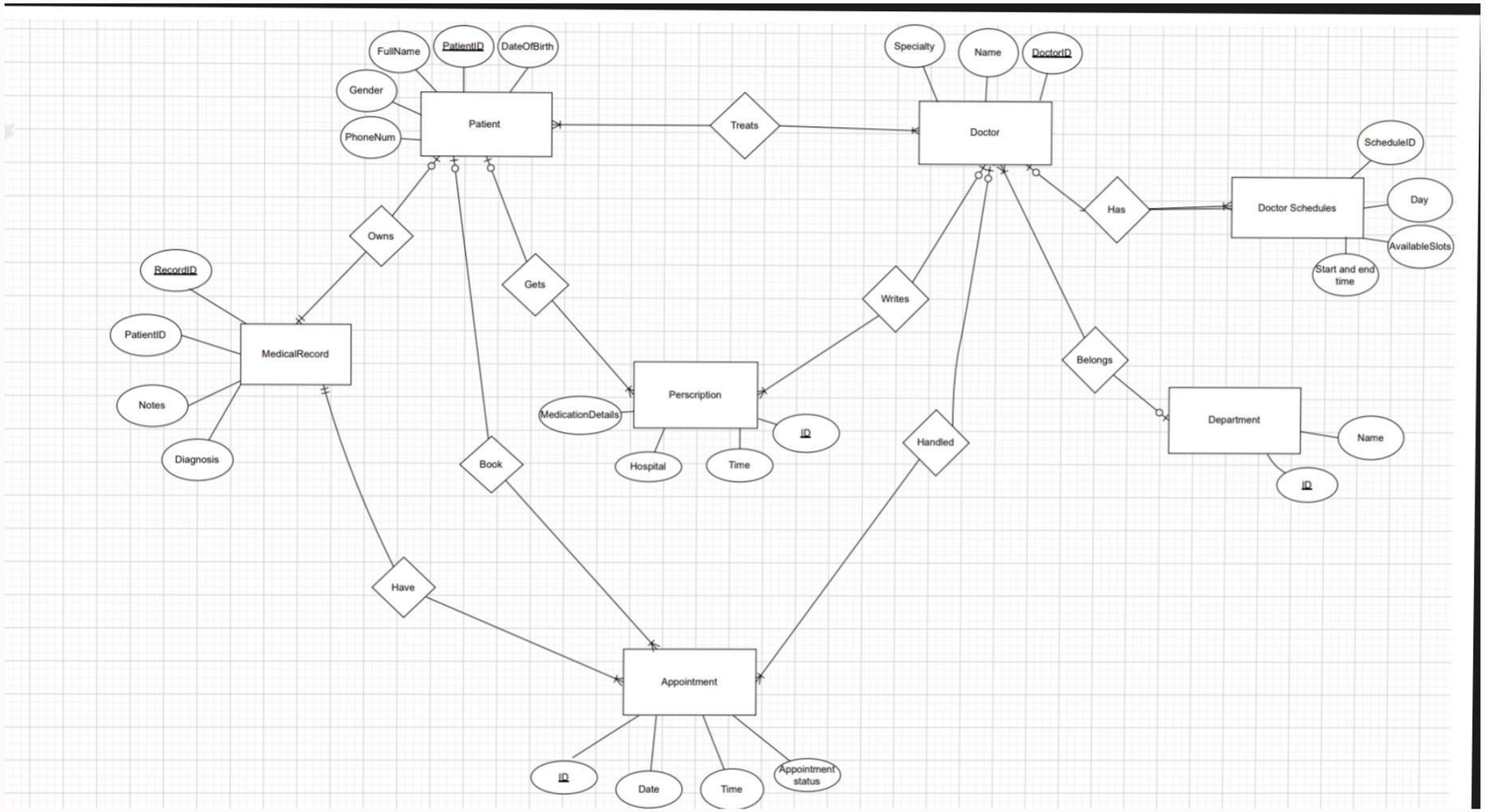
8. ScheduleDefines the availability of doctors during the week.

- Attributes:
  - ScheduleID (Primary Key)
  - DoctorID (Foreign Key → Doctor)
  - DayOfWeek
  - StartTime
  - EndTime

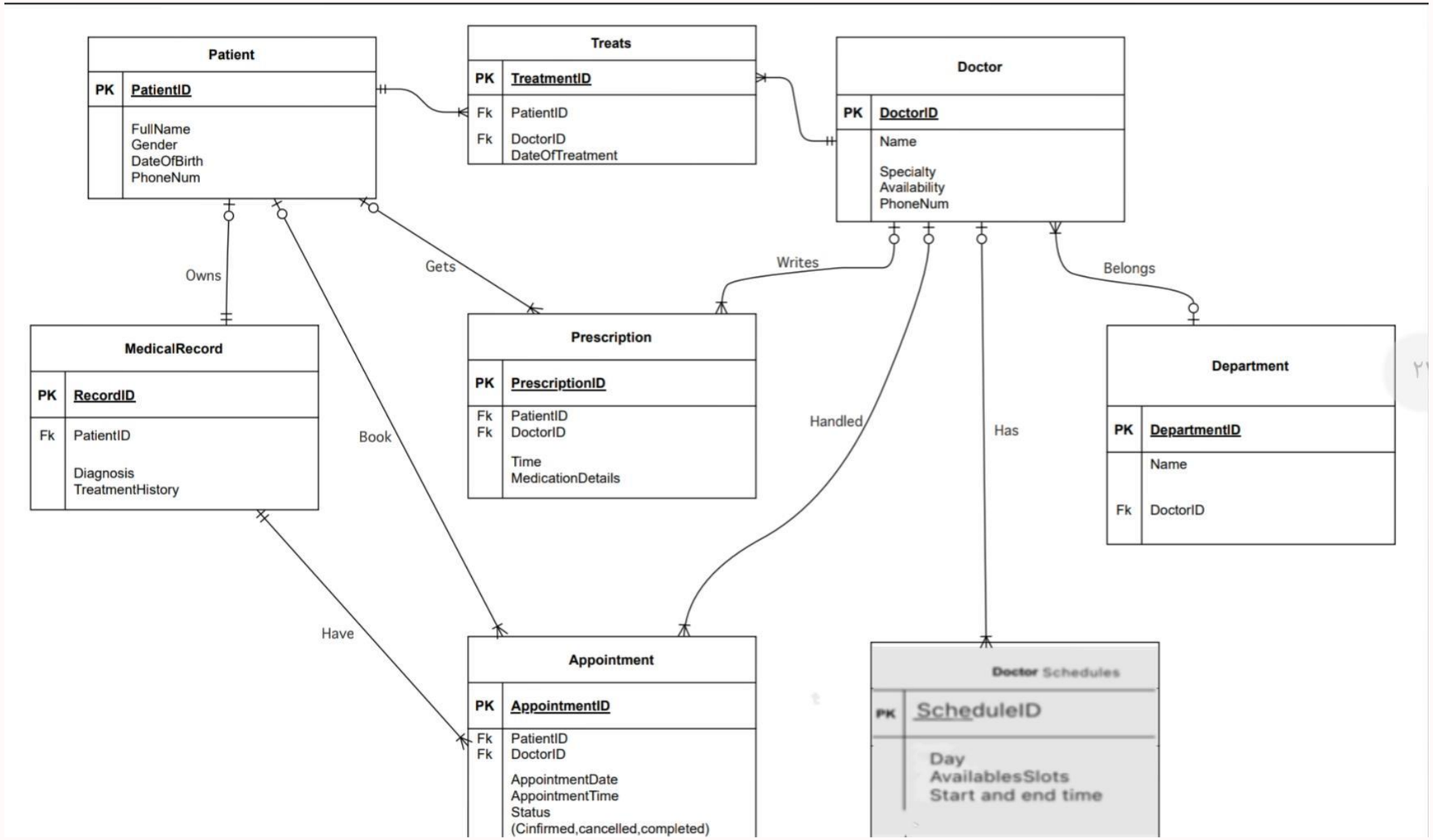
Entity Relationships (Overview)

- One Department can have many Doctors
- One Doctor can treat many Patients (via Treats)
- One Patient can have many Medical Records, Appointments, and Prescriptions
- One Doctor can have multiple Schedules and Appointments

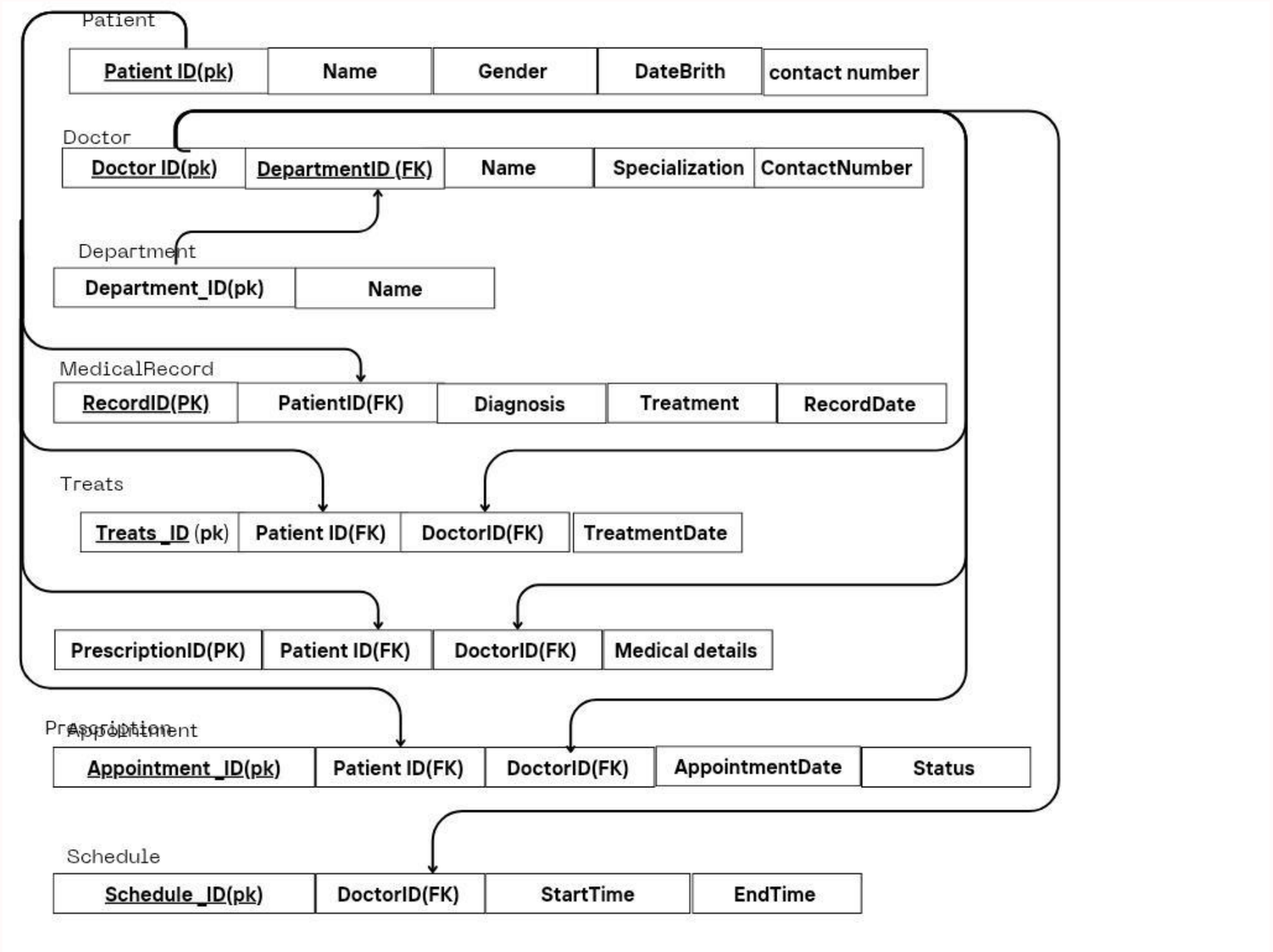
ERD


FullName
PatientID
DateOfBirth
Gender
PhoneNum
Patient
Treats
Specialty
Name
DoctorID
Doctor
ScheduleID
Has
Doctor Schedules
Day
AvailableSlots
Start and end time
Owns
RecordID
PatientID
Notes
Diagnosis
MedicalRecord
Gets
Writes
Belongs
Perscription
MedicationDetails
ID
Hospital
Time
Book
Handled
Department
Name
ID
Have
Appointment
ID
Date
Time
Appointment status

Logical Model



Relational Schema



Normalization

Normal Forms Explanation

🔶 First Normal Form (1NF)
- All attributes are atomic (single values).
- There are no repeating groups or multi-valued attributes.
- ✅ All tables satisfy 1NF.

🔶 Second Normal Form (2NF)
- All non-key attributes are fully functionally dependent on the entire primary key.
- Composite keys (e.g., if used) do not cause partial dependencies.
- ✅ All tables use single-column primary keys and meet 2NF.

🔶 Third Normal Form (3NF)
- There are no transitive dependencies — non-key attributes do not depend on other non-key attributes.
- All non-key attributes depend only on the primary key.
- ✅ Tables are properly decomposed to eliminate transitive dependencies.

Implementation

- Create Tables

```sql
-- Create Patient Table
CREATE TABLE Patient (
    PatientID NUMBER PRIMARY KEY,
    FullName VARCHAR2(100),
    Gender VARCHAR2(10),
    DateOfBirth DATE,
    PhoneNum VARCHAR2(15)
);

-- Create Doctor Table
CREATE TABLE Doctor (
    DoctorID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    Specialty VARCHAR2(50),
    Availability VARCHAR2(50),
    PhoneNum VARCHAR2(15)
);

-- Create Department Table
CREATE TABLE Department (
    DepartmentID NUMBER PRIMARY KEY,
    Name VARCHAR2(50),
    DoctorID NUMBER,
    CONSTRAINT fk_Department_Doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

-- Create MedicalRecord Table
CREATE TABLE MedicalRecord (
    RecordID NUMBER PRIMARY KEY,
    PatientID NUMBER,
    Diagnosis VARCHAR2(255),
    TreatmentHistory VARCHAR2(255),
    CONSTRAINT fk_MedicalRecord_Patient FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);
```

```
Table created.

Table created.

Table created.

Table created.
```

```sql

-- Create Treats Table
CREATE TABLE Treats (
    TreatmentID NUMBER PRIMARY KEY,
    PatientID NUMBER,
    DoctorID NUMBER,
    DateOfTreatment DATE,
    CONSTRAINT fk_Treats_Patient FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    CONSTRAINT fk_Treats_Doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

-- Create Prescription Table
CREATE TABLE Prescription (
    PrescriptionID NUMBER PRIMARY KEY,
    PatientID NUMBER,
    DoctorID NUMBER,
    Time DATE,
    MedicationDetails VARCHAR2(255),
    CONSTRAINT fk_Prescription_Patient FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    CONSTRAINT fk_Prescription_Doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

-- Create Appointment Table
CREATE TABLE Appointment (
    AppointmentID NUMBER PRIMARY KEY,
    PatientID NUMBER,
    DoctorID NUMBER,
    AppointmentDate DATE,
    AppointmentTime VARCHAR2(10),
    Status VARCHAR2(20), -- (Confirmed, Cancelled, Completed)
    CONSTRAINT fk_Appointment_Patient FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    CONSTRAINT fk_Appointment_Doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

-- Create Schedule Table
CREATE TABLE Schedule (
    ScheduleID NUMBER PRIMARY KEY,
    DoctorID NUMBER,
    Day VARCHAR2(20),
    AvailableSlots NUMBER,
    StartTime VARCHAR2(10),
    EndTime VARCHAR2(10),
    CONSTRAINT fk_Schedule_Doctor FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

```

```
Table created.

Table created.

Table created.

Table created.
```

Implementation

- Insert Data

-- Insert into Patient

```
1  -- Insert into Patient
2  INSERT INTO Patient VALUES (1, 'Fahad Al Saud', 'Male', TO_DATE('1990-05-15','YYYY-MM-DD'), '0551234567');
3  INSERT INTO Patient VALUES (2, 'Sara Al Qahtani', 'Female', TO_DATE('1988-09-20','YYYY-MM-DD'), '0557654321');
4  INSERT INTO Patient VALUES (3, 'Mohammed Al Harbi', 'Male', TO_DATE('2001-11-30','YYYY-MM-DD'), '0559876543');
5  INSERT INTO Patient VALUES (4, 'Noura Al Otaibi', 'Female', TO_DATE('1995-01-10','YYYY-MM-DD'), '0556789012');
6  INSERT INTO Patient VALUES (5, 'Abdullah Al Mutairi', 'Male', TO_DATE('1993-08-25','YYYY-MM-DD'), '0555678901');
7
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

-- Insert into Departments

```
15 -- Insert into Department
16 INSERT INTO Department VALUES (1, 'Cardiology', 1);
17 INSERT INTO Department VALUES (2, 'Dermatology', 2);
18 INSERT INTO Department VALUES (3, 'Neurology', 3);
19 INSERT INTO Department VALUES (4, 'General Medicine', 4);
20 INSERT INTO Department VALUES (5, 'Pediatrics', 5);
21
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

-- Insert into Doctor

```
8  -- Insert into Doctor
9  INSERT INTO Doctor VALUES (1, 'Dr. Khalid Al Zahrani', 'Cardiologist', 'MWF 9am-3pm', '0561234567');
10 INSERT INTO Doctor VALUES (2, 'Dr. Huda Al Dossary', 'Dermatologist', 'TTS 10am-4pm', '0569876543');
11 INSERT INTO Doctor VALUES (3, 'Dr. Ahmed Al Ghamdi', 'Neurologist', 'MWF 8am-2pm', '0567654321');
12 INSERT INTO Doctor VALUES (4, 'Dr. Fatimah Al Shammari', 'General Physician', 'All Week 9am-5pm', '0565555555');
13 INSERT INTO Doctor VALUES (5, 'Dr. Salem Al Amri', 'Pediatrician', 'MWF 10am-6pm', '0569998888');
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

-- Insert into MedicalRecord

```
22 -- Insert into MedicalRecord
23 INSERT INTO MedicalRecord VALUES (1, 1, 'Hypertension', 'Blood pressure control program');
24 INSERT INTO MedicalRecord VALUES (2, 2, 'Skin Allergy', 'Antihistamine treatment');
25 INSERT INTO MedicalRecord VALUES (3, 3, 'Migraine', 'Regular pain relief therapy');
26 INSERT INTO MedicalRecord VALUES (4, 4, 'Common Cold', 'Rest and hydration');
27 INSERT INTO MedicalRecord VALUES (5, 5, 'Asthma', 'Inhaler prescription and monitoring');
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

Implementation

- Insert Data

-- Insert into Treats

```
29  -- Insert into Treats
30  INSERT INTO Treats VALUES (1, 1, 1, TO_DATE('2025-04-01','YYYY-MM-DD'));
31  INSERT INTO Treats VALUES (2, 2, 2, TO_DATE('2025-04-02','YYYY-MM-DD'));
32  INSERT INTO Treats VALUES (3, 3, 3, TO_DATE('2025-04-03','YYYY-MM-DD'));
33  INSERT INTO Treats VALUES (4, 4, 4, TO_DATE('2025-04-04','YYYY-MM-DD'));
34  INSERT INTO Treats VALUES (5, 5, 5, TO_DATE('2025-04-05','YYYY-MM-DD'));
35
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

-- Insert into Appointment

```
36  -- Insert into Prescription
37  INSERT INTO Prescription VALUES (1, 1, 1, TO_DATE('2025-04-01 10:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Amlodipine
38  INSERT INTO Prescription VALUES (2, 2, 2, TO_DATE('2025-04-02 11:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Corticoster
39  INSERT INTO Prescription VALUES (3, 3, 3, TO_DATE('2025-04-03 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Sumatriptan
40  INSERT INTO Prescription VALUES (4, 4, 4, TO_DATE('2025-04-04 13:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Paracetamol
41  INSERT INTO Prescription VALUES (5, 5, 5, TO_DATE('2025-04-05 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Salbutamol
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

-- Insert into Prescription

```
36  -- Insert into Prescription
37  INSERT INTO Prescription VALUES (1, 1, 1, TO_DATE('2025-04-01 10:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Amlodipine
38  INSERT INTO Prescription VALUES (2, 2, 2, TO_DATE('2025-04-02 11:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Corticoster
39  INSERT INTO Prescription VALUES (3, 3, 3, TO_DATE('2025-04-03 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Sumatriptan
40  INSERT INTO Prescription VALUES (4, 4, 4, TO_DATE('2025-04-04 13:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Paracetamol
41  INSERT INTO Prescription VALUES (5, 5, 5, TO_DATE('2025-04-05 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'Salbutamol
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

-- Insert into Schedule

```
50  -- Insert into Schedule
51  INSERT INTO Schedule VALUES (1, 1, 'Monday', 5, '09:00', '15:00');
52  INSERT INTO Schedule VALUES (2, 2, 'Tuesday', 5, '10:00', '16:00');
53  INSERT INTO Schedule VALUES (3, 3, 'Wednesday', 4, '08:00', '14:00');
54  INSERT INTO Schedule VALUES (4, 4, 'Thursday', 6, '09:00', '17:00');
55  INSERT INTO Schedule VALUES (5, 5, 'Friday', 5, '10:00', '18:00');
56
```

```
1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.

1 row(s) inserted.
```

Implementation

- Queries

SELECT using WHERE and ORDER BY

```
-- List all patients who are Male, ordered by their name
```

```
1  -- List all patients who are Male, ordered by their name
2  SELECT PatientID, FullName, Gender, DateOfBirth
3  FROM Patient
4  WHERE Gender = 'Male'
5  ORDER BY FullName ASC;
6
```

PATIENTID	FULLNAME	GENDER	DATEOFBIRTH
5	Abdullah Al Mutairi	Male	25-AUG-93
1	Fahad Al Saud	Male	15-MAY-90
3	Mohammed Al Harbi	Male	30-NOV-01

Download CSV

3 rows selected.

SELECT using GROUP BY and Aggregate Function (COUNT)

```
-- Count number of patients treated by each doctor
```

```
1  -- Count number of patients treated by each doctor
2  SELECT DoctorID, COUNT(PatientID) AS TotalPatients
3  FROM Treats
4  GROUP BY DoctorID;
5
```

DOCTORID	TOTALPATIENTS
1	1
2	1
4	1
5	1
3	1

Implementation

- Queries

SELECT using JOIN between two tables

```
-- Show all appointments with patient and doctor names
```

```
-- Show all appointments with patient and doctor names
SELECT Appointment.AppointmentID,
       Patient.FullName AS PatientName,
       Doctor.Name AS DoctorName,
       Appointment.AppointmentTime,
       Appointment.Status
FROM Appointment
JOIN Patient ON Appointment.PatientID = Patient.PatientID
JOIN Doctor ON Appointment.DoctorID = Doctor.DoctorID;
```

APPOINTMENTID	PATIENTNAME	DOCTORNAME	APPOINTMENTTIME	STATUS
1	Fahad Al Saud	Dr. Khalid Al Zahrani	10:00 AM	Confirmed
2	Sara Al Qahtani	Dr. Huda Al Dossary	11:00 AM	Completed
3	Mohammed Al Harbi	Dr. Ahmed Al Ghamdi	12:00 PM	Cancelled
4	Noura Al Otaibi	Dr. Fatimah Al Shammari	01:00 PM	Confirmed
5	Abdullah Al Mutairi	Dr. Salem Al Amri	02:00 PM	Completed

SELECT using SUBQUERY

```
-- List all patients who have a prescription issued
```

```
-- List all patients who have a prescription issued
SELECT FullName
FROM Patient
WHERE PatientID IN (SELECT DISTINCT PatientID FROM Prescription);
```

FULLNAME
Fahad Al Saud
Sara Al Qahtani
Mohammed Al Harbi
Noura Al Otaibi
Abdullah Al Mutairi

```sql
CREATE OR REPLACE PROCEDURE GetPatientInfo (
    p_PatientID IN NUMBER,
    o_FullName OUT VARCHAR2,
    o_Gender OUT VARCHAR2,
    o_DateOfBirth OUT DATE,
    o_PhoneNum OUT VARCHAR2
)
IS
BEGIN
    SELECT FullName, Gender, DateOfBirth, PhoneNum
    INTO o_FullName, o_Gender, o_DateOfBirth, o_PhoneNum
    FROM Patient
    WHERE PatientID = p_PatientID;
END;
/
    -- Execution
DECLARE
    v_FullName Patient.FullName%TYPE;
    v_Gender Patient.Gender%TYPE;
    v_DOB Patient.DateOfBirth%TYPE;
    v_Phone Patient.PhoneNum%TYPE;
BEGIN
    GetPatientInfo(1, v_FullName, v_Gender, v_DOB, v_Phone);
    DBMS_OUTPUT.PUT_LINE('Patient: ' || v_FullName);
    DBMS_OUTPUT.PUT_LINE('Gender: ' || v_Gender);
    DBMS_OUTPUT.PUT_LINE('DOB: ' || TO_CHAR(v_DOB, 'DD-MON-YYYY'));
    DBMS_OUTPUT.PUT_LINE('Phone: ' || v_Phone);
END;
/
```

```
Procedure created.


Statement processed.
Patient: Fahad Al Saud
Gender: Male
DOB: 15-MAY-1990
Phone: 0551234567
```

Implementation

- Procedure

Stored Procedure 1:

- Get patient details based on patient ID

Implementation

- Procedure

Stored Procedure 2: UPDATE Patient Contact Number

```
CREATE OR REPLACE PROCEDURE UpdatePatientPhone (
    p_PatientID IN NUMBER,
    p_NewPhone IN VARCHAR2
)
IS
BEGIN
    UPDATE Patient
    SET PhoneNum = p_NewPhone
    WHERE PatientID = p_PatientID;
    COMMIT;
END;
/

BEGIN
    UpdatePatientPhone(1, '0551122334');
    DBMS_OUTPUT.PUT_LINE('Phone number updated successfully');
END;
/
```

```
Procedure created.

Statement processed.
Phone number updated successfully
```

Conclusion

This system is more than a digital tool; it’s a strategic solution.

It turns appointment challenges into efficiency and security opportunities.

Delivering a streamlined, reliable, and patient-focused medical environment.